ARC Training Centre for Behavioural Insights for Technology Adoption

ARC BITA Geek Seminars

Git + Python (Poetry): Hands-on Foundations

18 September 2025

Dr. Steve Bickley

s.bickley@qut.edu.au

https://arcbita.org/

Acknowledgement to Country

I acknowledge the **Turrbal** and **Yugara** as the First Nations owners of the lands where we now stand ***(Meanjin)***.

I pay respect to their Elders, lores, customs and creation spirits, and recognise that these lands have always been places of **teaching**, **research** and **learning**.

I acknowledge the important role Aboriginal and Torres Strait Islander people play within the QUT and wider community.

Welcome & Context

Welcome and thank you for your attendance today!

Slides Available at: https://github.com/stevejbickley/arcbita-geek-seminars

NOTE: I am *certainly not* a formal expert in this area. Much of my knowledge and approach have been bootstrapped over time, drawing on self-learning, experimentation, and piecing together insights/learnings and codes from different sources and experiences. As you can see in the recording, it often a messy/iterative process with significant trial and error (refer to discussion around 41 to 45 mins and again between Benno Torgler & Ben Chan about 1hr 06 min in the recording) to learn these tools (note: the hardest part is usually the install/setup, once you're up and running/indoctrinated/onboarded then the rate of learning & applying becomes much quicker!!), and this recording is a **perfect** example of such. With great flexibility in this tool, comes great responsibility (to prepare)!!

For more comprehensive guides/tutorials, see:

- https://www.python.org/about/gettingstarted/
- https://docs.python.org/3/tutorial/
- https://www.w3schools.com/python/python_getstarted.asp \

ChatGPT, Claude, Copilot, etc. are other great tools/resources for learning!! (again, the recording is a great example of the need to still slow down and double check/verify the model outputs.. ***especially*** before trying to do live/off script examples like this → refer to last 2 mins of recording for the crash and burn finish!! 🤣😭)

Why Python?

- Readable, beginner-friendly (??) syntax → faster onboarding (??) & fewer bugs
- Cross-platform (macOS/Windows/Linux) with strong IDE/editor support
- Large ecosystem (PyPI) for data, AI/ML, web, automation, and research
  - Also note, LLMs / ALMs also heavily geared towards Python code generation
- "Batteries included" standard library for everyday tasks
- *Tooling:* Poetry (deps/lockfiles), pytest (tests), Ruff/Black (style), …
- *Collaboration:* clear packaging, scripts, and CI/CD integration (continuous integration & continuous delivery/deployment)

Hint: Thank you **@Arian Mashhady** for the suggestion re: Github Copilot is free in VS Code (with some limits).

Might be worth to double check if free account = train on your data / code
(if you care about that..)

- **Python toolchain with pyenv + Poetry**
  - pyenv
    - Manages multiple Python versions per project/user.
  - Poetry
    - Dependency resolver, virtualenvs, scripts, build/publish.
  - Why together?
    - Pin Python version + lock dependencies for reproducibility.

- **IDEs:**
  - VS Code: https://code.visualstudio.com/docs/python/python-tutorial
  - PyCharm:
    - https://www.jetbrains.com/help/pycharm/quick-start-guide.html;
    - https://www.jetbrains.com/help/pycharm/configuring-python-interpreter.html

Image Ref: https://djangostars.com/blog/python-ide/

Why Git?

- Version history & safety
  - Roll back mistakes, experiment safely
- Branching & collaboration
  - Parallel work without conflicts (in theory!)
- PRs & review culture
  - Discuss changes, document decisions
- Portability & ecosystem
  - Works with GitHub/GitLab/Bitbucket → CI/CD pipelines
- Reproducibility
  - Lock project state alongside code

Core Python concepts

- **Interpreter & REPL** (Read-Eval-Print Loop – run one line at a time) **vs running scripts** (python file.py)
- **Modules, packages & imports w/ entry points** via e.g. pyproject.toml scripts
- **Virtual environments & dependency locking** (e.g. Poetry + poetry.lock)
- **Types & collections:** int/float/str/bool/None, list/tuple/dict/set
- **Control flow & iteration:** if/elif/else, for/while, comprehensions
- **Functions & scope** (local/enclosing/global/built-in) + type hints (def f(x: int) -> str)
- **Classes & dataclasses** for lightweight models + dunder or double underscore methods (e.g., __init__, __str__, __len__, __getitem__)
  - **Key difference:** data classes are for lightweight "records/containers," while regular classes are more general and flexible (for logic, behaviour, inheritance, etc.)
- **Exceptions & testing:** try/except, pytest

```
squares = []
for x in range(5):
    squares.append(x**2)
```

=

```
squares = [x**2 for x in range(5)]
```

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

`__init__`: runs when you create an object.
`__str__`: defines how an object prints.
`__len__`: lets you use `len(obj)`.
`__getitem__`: makes objects indexable like lists.

Core Git concepts

- **Repository**
  - Local (lives on your machine) vs remote (hosted on e.g. GitHub)
  - Workflow: you commit locally, then git push to share your changes with the remote repo. To bring in others' changes, you git pull (fetch + merge)
- **Commit**
  - A snapshot of your project at a point in time, saved w/ a message
  - *Best practice:* make commits small, logical, and purposeful (e.g. "fix bug in login validation" instead of "misc changes")
- **Branch**
  - A **branch** is a movable pointer to a commit, used to isolate work
  - E.g., main/ or master/ (production-ready), dev/ (integration/testing), feature/* (one branch per new feature)
- **Remote / forked workflows**
  - **Origin:** By convention, the name of your *own remote copy* of a repo (the one you cloned from or your personal fork)
  - **Upstream:** The *main project's repository*, usually the original repo you forked from. You add it separately so you can keep your fork in sync
  - Typical flow in forked workflows:
    - Pull changes from upstream → merge into your local main branch
    - Push that updated main branch to your origin
    - Create or switch to a feature branch → push it to origin
    - Open a Pull Request from your feature branch into the upstream project.
- **Pull Request (PR)**
  - A PR is a request to merge your branch into another (often main)
  - Allows discussion, review, and automated checks before changes go in
  - Think of it as the *collaborative gate* where others can review your work

Typical repo workflow

- Clone
  - git clone <starter-repo-url>
- Create branch
  - git checkout -b feature/hello-poetry
- Make change
  - Edit files, run tests.
- Commit & push
  - git add -A or git add .
  - git commit -m "feat: initial demo"
  - git push -u origin feature/hello-poetry
- Open PR
  - Review → merge

Hands-on Foundations *(Installs)*

Feel free to just observe/follow along and watch the recording later

Slides Available at: https://github.com/stevejbickley/arcbita-geek-seminars

First, always check your toolchain!

```
# Versions (any platform e.g., Pycharm, terminal/command line, etc.)
git --version
python --version
poetry --version   # after install
pyenv --version    # after install
```

Don’t worry.. we will install together if missing!

Slides Available at: https://github.com/stevejbickley/arcbita-geek-seminars

Install Git, pyenv & Poetry

Python → https://www.python.org/downloads/

- First step in most cases, to install python itself. Even after (or before) installing the IDEs, you generally need to install python.
- Follow through with the installation guide/wizard. After you've installed then close the terminal/command line and re-open then verify the installation was successful with `python --version`

Git → https://git-scm.com/downloads

- **macOS**: brew install git (if you don't already have homebrew installed on your system then follow the installation instructions at https://brew.sh/)
- **Windows**: Download installer from https://git-scm.com/downloads, after downloading the .exe file then click & follow through the install wizard, you can mostly keep the default options but be sure to enable *"Git from the command line/terminal"* during setup (sometimes also called "Add git to PATH", this is so you can call git from anywhere on your system).
- **Linux**: apt-get install git (Debian/Ubuntu) or yum install git (Fedora/RHEL) → https://git-scm.com/downloads/linux

Pyenv → https://github.com/pyenv/pyenv

- **macOS (Homebrew)**:
  - brew update
  - brew install pyenv
- **Windows (pyenv-win)**: https://pypi.org/project/pyenv-win/
  - pip install pyenv-win; ensure PATH updated; restart terminal
- **Linux**:
  - curl -fsSL https://pyenv.run | bash

Poetry → https://python-poetry.org/docs/

- **macOS/Linux**: curl -sSL https://install.python-poetry.org | python3 -
- **Windows (PowerShell)**: (Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | py -
- **Recommended**: keep virtualenvs in-project → poetry config virtualenvs.in-project true

pyenv handles Python version management/installs during the session:

pyenv install <version>

Version naming

- Use the exact version string available (pyenv install --list shows all). Sometimes 3.11.9 vs 3.11.10 matters.

Troubleshooting: Common Errors & Quick Hits

- **PATH not updated?**
  - Close/reopen terminal/shell/command line
- **Multiple Pythons?**
  - pyenv which python; poetry env info
- **Poetry not found?**
  - Check installer output; add to PATH; restart terminal/shell/command line
- **Windows: Git Bash vs PowerShell confusion**
  - Reminder to run commands in the same shell you set up with (PATH differences can bite)
- **Permission errors on Windows (admin rights)**
  - Especially for QUT laptops, flag that *makemeadmin* might need to be active when installing/updating
- **Proxy/firewall issues on institution networks**
  - Curl/PowerShell installers can fail if behind a proxy.. some users may need to retry on home Wi-Fi

Hands-on Foundations *(Basics)*

Feel free to just observe/follow along and watch the recording later

Slides Available at: https://github.com/stevejbickley/arcbita-geek-seminars

Git Clone Methods (HTTPS vs SSH)

Note: Refer to approx. 1hr 18 mins to 1hr 21 mins for example of a new git repo setup and the process of cloning a repo (using HTTP method)

HTTPS (default, easier):

```
git clone https://github.com/user/repo.git
```

→ **Note:** it can get annoying when you're pushing back to the remote repo to have to enter login details each time (which is why SSH can be good!)
→ Refer to recording at approx. 1 hr 22 mins to 1 hr 24 mins of recording to see the annoyance in real time & evidence on why you should probably not cut corners and just go for SSH setup!!

SSH (requires SSH key setup):

1. Generate SSH key: ssh-keygen -t ed25519 -C "your_email@example.com"
2. Add key to ssh-agent & copy to GitHub profile (Settings → SSH and GPG keys)
3. Ensure key path added to your PATH/agent
4. Clone with: git clone git@github.com:user/repo.git

Refer to:

→ https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent
→ https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account

GitHub CLI (optional):

```
gh repo clone user/repo
```

Typical Git Workflow & .gitignore hygiene

Typical workflow

- Clone
  - git clone <starter-repo-url>
- Create branch
  - git checkout -b feature/hello-poetry
- Make change
  - Edit files, run tests.
- Commit & push
  - git add -A or git add . *(My preferred.. Lazy!)*
  - git commit -m "feat: initial demo"
  - git push -u origin feature/hello-poetry
- Open PR
  - Review → merge

** Refer to recording at approx. 1 hr 21 mins to 1 hr 24 mins to see the process of git clone, git checkout, make changes, commit, and a partial/incomplete push (I don't remember my password..)

Add .gitignore file to your repo

```
# Python
__pycache__/
*.py[cod]
*.egg-info/

# Poetry virtualenv in project
.venv/

# OS noise
.DS_Store
Thumbs.db
```

*** Refer to recording at approx. 1:14 for discussion of the use of .gitignore → protect your secrets!!*

Merge vs rebase

- Merge
  - Keeps full history; creates merge commit
- Rebase
  - Linear history; rewrites commits onto target branch
- Rule of thumb
  - Prefer merge for teams
  - rebase local feature branches before PR if repo policy allows

Resolve simple conflicts (via PR)

- Pull latest main → update your branch
- Fix conflicts in editor
  - (markers <<<<<<< ======= >>>>>>>)
- Re-test, commit, push; request review

Create project with Poetry

```
poetry new hello-poetry          OR          poetry add hello-poetry          OR          poetry init
```

```
cd hello-poetry
pyenv install 3.11.9      # example version
pyenv local 3.11.9
poetry env use python      # bind Poetry to local Python
poetry add requests        # add a dependency

# Add a script (pyproject.toml)
[tool.poetry.scripts]
hello = "hello_poetry.__main__:main”
```

```
# activate environment
poetry shell # OR
source .venv/bin/activate

# Then poetry add…
```

Then we can run & test:

```
poetry run python -m hello_poetry
poetry run pytest          # if tests added
poetry add pytest --group dev
poetry lock                # ensure lockfile present
```

Refer to approx. 51 to 53 mins in recording for working example of ***successful*** *poetry env activation (using an old environment I had already setup/prepared before the session!!).*

Refer to approx. 1hr 24 mins for an successful creation of new poetry environment, coupled with an unsuccessful poetry env activation. Here I could have used the “poetry shell” command (see above) instead of the commands I was trying to use..

Wrap up & Next steps

Thank you again for your attendance today!

Slides Available at: https://github.com/stevejbickley/arcbita-geek-seminars

What we achieved during the recording

- Background/intro/context to Python & Git
- Install git, pyenv, poetry using your IDE of choice (e.g. PyCharm, Visual Studio)
- Clone starter repo and create new branch
- Create ~~or edit~~ a small script/test
- Commit ~~and push + open a PR~~
- ~~Resolve a simple conflict and merge~~
- Poetry: create new environment, load environment, add a dependency, run script with poetry run